

Toward a Sparse and Interpretable Audio Codec

John Vinyard

Austin TX, USA

john.vinyard@gmail.com

Abstract—Most widely-used modern audio codecs, such as Ogg Vorbis and MP3, as well as more recent “neural” codecs like Meta’s Encodec [1] or the Descript Audio Codec [2] based on block-coding; audio is divided into overlapping, fixed-size “frames” which are then compressed. While they often yield excellent reproduction quality and can be used for downstream tasks such as text-to-audio, they do not produce an intuitive, directly-interpretable representation. In this work, we introduce a proof-of-concept audio encoder that represents audio as a sparse set of events and their times-of-occurrence. Rudimentary physics-based assumptions are used to model attack and the physical resonance of both the instrument being played and the room in which a performance occurs, hopefully encouraging a sparse, parsimonious, and easy-to-interpret representation.

1. INTRODUCTION

Codecs are commonly thought of as transformations into a compressive representation, and while compression is very important, codecs might also transform signals into a representation that is easier to interpret and to manipulate.

We imagine a future audio codec where some types of musical composition could take place directly in the audio codec space. The text-to-audio paradigm works well for non-musicians creating background music for movies, advertisements or social media content, but it is our view that experienced musicians and composers work in a “space” not fully captured by language and that they will prefer a much finer-grained representation that provides multiple scales of interpretability and control.

This work does not seek to produce a generative model of audio, but could serve as the underlying encoding on which generative models are trained. It is the authors’ intuition that models trained on this quasi-symbolic representation might have a much more thorough “understanding” of the content they produce, given the point-cloud-like nature of the signal. Instead of predicting the next arbitrarily-sized frame, the generative model would be predicting the relationships between “events”. Long-term coherence in musical generation continues to be a challenge, and we speculate that many models spend large shares of their capacity learning to reproduce physical resonance, reducing the share that may model the human forces that drive them in interesting, musical directions.

All model and experiment code is implemented using the PyTorch [3] Python library and can be found on GitHub¹. Audio examples can be heard at this <https> URL²

2. PRIOR WORK

We take inspiration from a wealth of previous works dealing with sparse and interpretable representations of audio signals. Matching pursuit [4] is an iterative algorithm that decomposes a signal into a sparse set of representative “atoms”. It produces a representation of audio signals similar to the one used in granular synthesis [5], which represents audio as point-cloud-like structure of audio “grains” arranged in time. While the notion of “events in time” feels intuitive

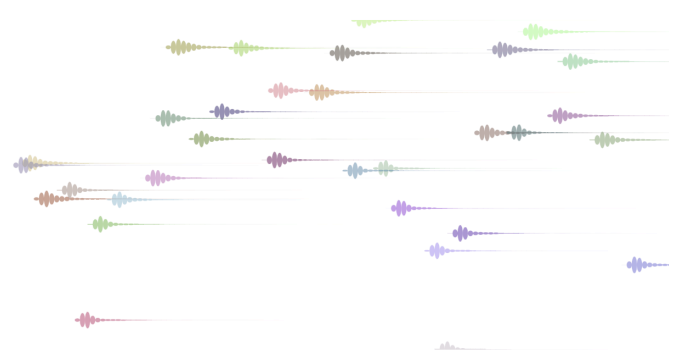


Fig. 1: In this visualization of the codec representation, we see that events can overlap and vary in length. Time is along the x-axis. Event positions along the y-axis and event colors are chosen by applying t-SNE [10] to the set of 32 event vectors, targeting a single dimension for the y-axis and three dimensions to represent an RGB color value.

to us, both approaches traditionally require very large numbers of “atoms” or “grains” that are often orders of magnitude smaller, or shorter than our typical conception of audio events. Furthermore, matching pursuit, at least when operating naively in the time domain can spend capacity on perceptually irrelevant details and struggle to reproduce noisy signals in a convincing way. In recent years, attempts have been made to use longer atoms and parameterize dictionaries of atoms in more meaningful ways [6]. In this work we hope to continue the trend toward fewer and more meaningful “atoms” or “events” that exist at scales native to say, a human musician.

Unsupervised audio source separation, such as in [7] seeks to separate mixtures of instruments or voices into distinct tracks, or “stems” as they are often referred to in music production, while unsupervised music transcription [8] seeks to infer a musical score, or set of instructions sufficient for a musician to reproduce the piece. Other recent works seek to produce meaningful latent representations by encouraging sparsity as part of the training objective [9]. In this work, we seek to infer a score of sorts, where each encoded note or event contains enough information to create a perceptually indistinguishable reproduction of the event.

DDSP - source-excitation models (NEEDS A REFERENCE)

3. AUDIO CODEC DETAILS

Our proposed audio encoding is simply a set of two-tuples, each consisting a scalar event time, and the parameters required for the decoder to render the event, transforming it into “raw” PCM audio samples. In addition to straightforward seeking and slicing, this encoding also makes it possible to filter a subset of events in a particular audio slice according to some criteria, an operation that is less straightforward in a block-coding scheme.

3.1. Compression Rate

While a high compression rate is not the primary focus of this work, a compressive representation is important and often indicative of

¹<https://github.com/JohnVinyard/matching-pursuit>

²<https://blog.cochlea.xyz/sparse-interpretable-audio-codec-paper.html>

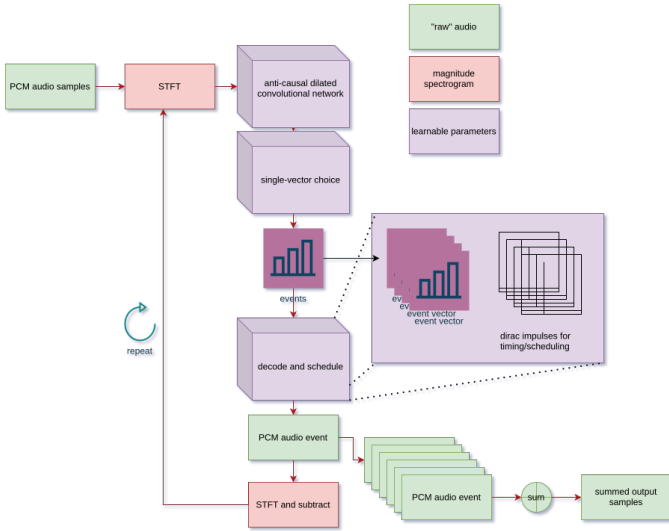


Fig. 2: High-Level Model Architecture. The encoder and decoder work together to incrementally remove energy from the input representation, and STFT-based magnitude spectrogram, in our case. At each step, the encoder produces a single event vector and time-of-occurrence. The decoder generates an audio event, “schedules” it, and subtracts it from the input representation.

generalization?!?! For our experiment, the encoder is run a fixed number of steps, 32 in our case, and produces a two-tuple of time-of-occurrence and event vector which has 32 dimensions. Each event time can be stored as a single scalar value. Because we encode 2^{16} samples, or around 5.94 seconds of audio at 22050hz, we arrive at:

4. MODEL

4.1. Audio Representation

In this work, we seek an audio representation that can be iteratively decomposed, without encoding perceptually-irrelevant details. For example, a model using “raw” PCM samples as its representation might struggle to remove noisy signals due to undue focus on perceptually irrelevant details.

4.2. Encoder

Inspired by the iterative matching pursuit algorithm [4], the encoding process is iterative, or incremental. At each step, the encoder chooses a single event vector and time-of-occurrence and passes this to the decoder, which generates and schedules audio and subtracts it from the input representation. This process is repeated for some fixed number of steps or until some stopping condition is reached.

In this work, the encoder is parameterized as an anti-causal, dilated convolutional network, but other types of network, such as UNets or Transformers may yield superior results and will be explored in future explorations.

4.3. Streaming Algorithm

4.4. Decoder

In order to encourage a sparser and easier-to-interpret representation, we parameterize the decoder as a neural network with the following structure:

Each event is roughly interpreted as a physical process, including the injection of energy into some system and the long convolutions representing the resonance of the system and the way the system is deformed over time as it resonates.

TODO: Diagram of decoder block and scheduling process

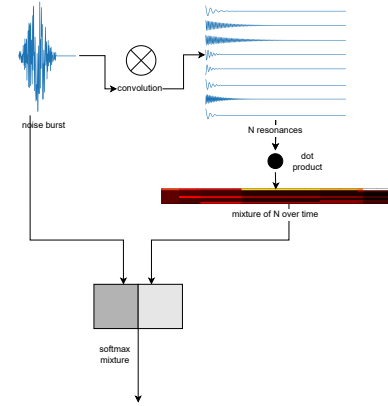


Fig. 3: Decoder Block

5. EXPERIMENTS

5.1. Data Set

We train our model on the MusicNet dataset [11], an open dataset containing 33 hours of classical music, recorded in diverse spaces using a range of recording equipment of varying quality. The recordings often include leading silence, trailing applause, and incidental human sounds throughout (coughs, movement, etc).

5.2. Training Algorithm

During training, we repeatedly draw segments of audio that are approximately 5.94 seconds in length from the Music Net dataset at random, with length 2^{17} samples at 22050hz sampling rate and a batch size of two. Each batch is passed through the encoding and decoding process for a fixed number of steps, 32, in our case, and then an iterative loss is computed, encouraging each step to have removed as much energy (expressed as the L1 norm) from the signal as possible. This loss is minimized using the Adam optimizer with a learning rate of $1e^{-4}$.

5.3. Loss Functions

The loss function is also iterative and greedy. Using the same magnitude spectrogram representation observed by the encoder, the loss function attempts to maximize the reduction in the L1 norm of the spectrogram at each step. Prior to computing the loss, the input events are sorted in order of descending norm, such that the event with the most energy is subtracted first and the event with the least energy is subtracted last.

6. INTERPRETABLE AND MANIPULATABLE REPRESENTATION

6.1. Events and Times-of-Occurrence

At the coarse-grained level, overall information density can be gleaned by viewing event times, without regard with event “contents”.

6.2. Decoder Interpretation

For the decoder used in this set of experiments, intermediate states of the decoder are designed to be meaningful given a source-excitation interpretation of the synthesis process.



Fig. 4: Here we can see intermediate stages of the source-excitation-based decoder at work. From left to right, we see A. a spectrogram of the initial noisy impulse B. a spectrogram of the noisy impulse convolved with the chosen decaying resonance and C. the result from step B. convolved with a room impulse response.

7. FUTURE WORK

7.1. Better Perceptual Losses

TODO: Reference to textural loss paper from Josh McDermott

7.2. Decoder Variants

7.3. Redundant Events and Sparsity

It’s possible that other decoder variants may yield better results, and that events might be best routed to different specialized decoders.

8. EQUATIONS

Equations should be placed on separate lines and consecutively numbered with equation numbers in parentheses flush with the right margin, as illustrated in (1) that gives the homogeneous acoustic wave equation in Cartesian coordinates [12],

$$\Delta^2 p(\mathbf{x}, t) - \frac{1}{c^2} \frac{\partial^2 p(\mathbf{x}, t)}{\partial t^2} = 0, \quad (1)$$

where $p(\mathbf{x}, t)$ is an infinitesimal variation of acoustic pressure from its equilibrium value at position $\mathbf{x} \in \mathbb{R}^3$ and time t , and where c denotes the speed of sound.

Symbols in your equation should be defined before the equation appears or immediately following. Use (1), not Eq. (1) or equation (1), except at the beginning of a sentence: “Equation (1) is ...”

REFERENCES

- [1] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, “High fidelity neural audio compression,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.13438>
- [2] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, “High-fidelity audio compression with improved rvqgan,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.06546>
- [3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems* 32. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [4] S. Mallat and Z. Zhang, “Matching pursuits with time-frequency dictionaries,” USA, Tech. Rep., 1993.
- [5] I. Xenakis, *Formalized Music: Thought and Mathematics in Composition*. Bloomington: Indiana University Press, 1971.
- [6] J. Alfonso, “Reds: A new asymmetric atom for sparse audio decomposition,” *DAFx17*, 2017. [Online]. Available: https://www.dafx17.eea.ed.ac.uk/papers/DAFx17_paper_66.pdf
- [7] K. Schulze-Forster, G. Richard, L. Kelley, C. S. J. Doire, and R. Badeau, “Unsupervised music source separation using differentiable parametric source models,” 2023. [Online]. Available: <https://arxiv.org/abs/2201.09592>
- [8] T. Berg-Kirkpatrick, J. Andreas, and D. Klein, “Unsupervised transcription of piano music,” in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27. Curran Associates, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/file/e51e118b28def0cabd5237b4e1fec499-Paper.pdf
- [9] M. Lisboa and G. Bellec, “Spiking music: Audio compression with event based auto-encoders,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.01571>
- [10] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008. [Online]. Available: <http://www.jmlr.org/papers/v9/vandermaaten08a.html>
- [11] J. Thickstun, Z. Harchaoui, and S. M. Kakade, “Learning features of music from scratch,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [12] E. Williams, *Fourier Acoustics: Sound Radiation and Nearfield Acoustic Holography*. London, UK: Academic Press, 1999.